UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY (FET)

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

PROJECT TITTLE

TASK 5 REPORT

UI Design and Implementation

CrowdSenseNet - App Identity, Visual Design and Frontend Implementation

COURSE TITLE: **INTERNET PROGRAMMING AND MOBILE PROGRAMMING**

COURSE CODE: **CEF 440**

COURSE INSTRUCTOR**: Dr. Nkameni Valery**

ACADEMIC YEAR 2025 - 2026

Project Group: Group 13

Submitted by:
1. Imele Azafa Jose de Kajos …… **FE23A068**
2. Ayuk Amelia Agbor Orock ……..…… **FE23A020**
3. Ngome Fernandez Manyaka …… **FE23A111**
4. Tadjeu Ngnintedem Aubry-Junex ……..…… **FE23A148**
5. Bessong Takang Tabot Willy ……..…… **FE23A025**

Member Name	**Role / Section**
Amelia	App Identity and Branding
Jose	Visual Design (Figma)
Aubry	Home, Coverage Map Screen UI and Frontend Integration
Fernandez	Settings, History Screen and Frontend Integration
Bessong	……..

Table of Contents

1. Introduction

This report documents Task 5 of the CrowdSenseNet project, covering the full UI design and implementation work completed by each member of the group. CrowdSenseNet is an Android crowdsensing application that collects mobile network signal measurements from smartphones and uses machine learning to predict coverage holes across Cameroon.

The task was divided into five parts. Each member was responsible for a specific section of the application interface. This report covers the following areas:

- Amelia: App identity and branding
- Jose: Home Screen UI
- Aubry: Coverage Map Screen
- Bessong: History Screen
- Fernandez: Settings Screen and Frontend Integration

Note on Contribution: Bessong did not complete his assigned work on the History Screen. No design file, no implementation file, and no screenshot were submitted. His section is included in this report for completeness, but the content presented there was reconstructed by the group based on the requirements described in the task brief, not on any work he actually produced.

2. Amelia - App Identity and Branding

2.1 Introduction

App identity is the visual and conceptual personality of a mobile application. It communicates who the app is, what it does, and what values it stands for before the user even opens it. A well-designed app identity builds trust, creates recognition, and gives the application a professional presence that reflects the quality of the work behind it.

This section documents the complete app identity for CrowdSenseNet, an Android mobile crowdsensing application that collects cellular network signal measurements from ordinary smartphones and uses machine learning to predict mobile network coverage holes across Cameroon. The identity covers the app name, the official logo and its meaning, the chosen tagline, the complete color palette, the typography system, and the design principles that guide every visual decision in the application.

2.2 App Name

Element	Details
Full Name	CrowdSenseNet
Short Name	CSN
Category	Network Monitoring / Mobile Crowdsensing
Platform	Android (Mobile Application)
Target Region	Cameroon, Central Africa

The name CrowdSenseNet is built from three words, each carrying a specific meaning:

- Crowd: represents the community of ordinary smartphone users who contribute signal data. The power of the system comes from many people working together, not one expensive piece of equipment.
- Sense: refers to the sensing capability of those smartphones. Each phone acts as a sensor, reading RSRP, RSRQ, SINR, RSSI, Cell ID, and GPS location from the environment around it.
- Net: refers both to the cellular network being measured and to the interconnected network of users collaborating to build a complete coverage picture across Cameroon.

Together the name says exactly what the app does in one word: a crowd of people sensing the network together.

2.3 Logo

2.3.1 Official Logo

The official logo for CrowdSenseNet was designed to communicate the three core ideas of the application in a single visual: location, connectivity, and intelligence.

Figure 1: Official CrowdSenseNet App Icon / Logo

2.3.2 Logo Element Breakdown

The logo is made up of three distinct visual elements that work together to tell the story of the application:

2.3.3 The Map Pin

The dominant shape of the logo is a map location pin, which is the universal symbol for a geographic location. This choice is deliberate and meaningful. CrowdSenseNet is fundamentally a location-based application. Every signal measurement it collects is tied to a specific GPS coordinate. The coverage map it produces is a geographic product. The map pin immediately tells any user that this app is about knowing where things are happening on the ground.

The pin is rendered in a smooth gradient that flows from bright cyan at the top to deep royal blue at the bottom. This gradient gives the icon a sense of depth and dimension, making it visually rich even at small sizes. The rounded circular head of the pin and the sharp downward point at the base follow the standard map pin shape that users around the world already recognise from Google Maps and other navigation tools.

2.3.4 The Network Mesh

Inside the circular head of the map pin sits a network mesh, which is a pattern of nodes connected by lines forming an irregular polygon. This mesh represents two things at once. First, it represents the cellular network itself, meaning the web of towers, connections, and signal paths that CrowdSenseNet is designed to measure and map. Second, it represents the machine learning intelligence that the system applies to the collected data, which is a connected graph of data points from which patterns and predictions emerge.

The mesh nodes are rendered in two shades: deep navy blue circles and lighter cyan circles, creating a visual contrast that suggests different data points collected by different users. The lines connecting them represent the relationships and patterns the ML model discovers in the data.

2.3.5 The WiFi Signal Waves

Above the map pin, three concentric signal waves arc upward, rendered in the same teal-to-blue gradient as the pin body. These waves are immediately recognisable as a WiFi or cellular signal symbol. They communicate the core function of the application at a glance: this app is about measuring and mapping signal.

The three waves also carry symbolic meaning. The smallest wave closest to the pin represents a weak or local signal. The middle wave represents average coverage. The largest outermost wave represents strong, wide-area coverage. Together they visually echo the three coverage classes the application uses: good, average, and poor.

2.3.6 Logo Meaning Summary

Logo Element	Visual Description	What It Represents
Map Pin	Large teardrop location pin shape with teal-to-blue gradient	Geographic location. Every measurement is tied to a GPS coordinate.
Network Mesh	Polygon of connected nodes inside the pin head	The cellular network being measured and the ML intelligence analysing the data.
WiFi Signal Waves	Three concentric arcs above the pin	Cellular signal sensing. The three waves echo the three coverage classes.

Teal-to-Blue Gradient	Smooth colour transition from #00BCD4 to #1565C0	Technology, trust, intelligence, and depth.
White Inner Circle	White ring separating the pin outline from the mesh	Clarity and transparency.

2.3.7 Logo Usage Guidelines

- The logo must always appear on a white or very light background for maximum contrast and visibility.
- The minimum display size is 48 x 48 pixels. Below this size the mesh detail becomes invisible and the logo loses meaning.
- The logo must never be stretched, rotated, recoloured, or have any element removed.
- A minimum clear space equal to the width of one signal wave must be maintained around the logo at all times.
- For the Android app icon, the logo is placed on a white background inside a rounded square following the Android Adaptive Icon specification.
- For print or presentation use, the logo should be displayed at a minimum of 2cm wide.

2.4 Tagline

"Your phone. Your network. Your map."

This tagline was chosen because it speaks directly to the user in plain, personal language. Each of the three short sentences carries a specific meaning:

- Your phone: the device already in your pocket is all you need. No special equipment, no technical training, no extra cost.
- Your network: the signal quality you live with every day is what we measure. You experience the problem. You help document it.
- Your map: the coverage map that results belongs to the community. Everyone who contributes data benefits from seeing where the gaps are.

Tagline	Context / Where Used
Your phone. Your network. Your map.	Primary tagline: app store listing, onboarding screen, splash screen, and marketing materials
Mapping Cameroon. One signal at a time.	Secondary tagline: academic reports and presentations emphasising the geographic mission

Coverage Prediction Platform	Sub-brand descriptor: used in technical documentation and the admin dashboard

2.5 Color Palette

The color palette was derived directly from the logo. The teal-to-blue gradient of the map pin and signal waves defines the brand colors of CrowdSenseNet. Every screen in the application uses these same colors to create a consistent and professional visual experience.

2.5.1 Brand Colors

Color Name	Hex Code	Role	Where It Is Used
Deep Royal Blue	#1565C0	Primary	Navigation bar, primary buttons, screen headers, active states
Teal / Cyan	#00BCD4	Secondary	Accent elements, signal indicators, icon highlights, card borders
Gradient Mid Blue	#0077B6	Supporting	Gradient fills, section dividers, secondary buttons
White	#FFFFFF	Background	Screen backgrounds, card surfaces, input fields, inner icon ring
Dark Slate	#263238	Text	Body text, labels, headings, captions
Light Blue Grey	#ECEFF1	Surface	Card backgrounds, list items, bottom sheet backgrounds

2.5.2 Coverage Heatmap Colors

Color	Hex Code	Coverage Class	RSRP Threshold	Meaning

Signal Green	#4CAF50	Good Coverage	RSRP > -80 dBm	Strong signal. Excellent user experience expected.
Amber	#FF9800	Average Coverage	RSRP -80 to -100 dBm	Acceptable signal. Some degradation possible.
Danger Red	#F44336	Coverage Hole	RSRP < -110 dBm	Very weak signal. Coverage hole confirmed.
Neutral Grey	#9E9E9E	Insufficient Data	< 50 readings per 1 km2	Not enough data yet to generate a prediction.

2.6 Typography

Typography shapes how the application feels to use. Consistent type choices create visual hierarchy, guide the user's eye through the interface, and establish the professional tone of the application. CrowdSenseNet uses Roboto, which is Google's official typeface for Android and the default font in Material Design 3. It was designed specifically for digital screens and performs well at all sizes from large display text down to small captions.

Level	Weight	Size	Where Used
Display	Roboto Bold	32sp	App name on splash screen and onboarding
Headline	Roboto Bold	24sp	Screen titles: Home, Map, History, Settings
Title	Roboto Medium	20sp	Section headers, card titles, dialog titles
Signal Values	Roboto Medium	18sp	Live RSRP, RSRQ, SINR, RSSI values on dashboard

Body Large	Roboto Regular	16sp	Main descriptive text, instructions, session stats
Body Small	Roboto Regular	14sp	Secondary labels, sub-descriptions, list items
Caption	Roboto Light	12sp	Timestamps, map zone labels, chart axis labels

2.7 Design Principles

2.7.1 Material Design 3

The application follows Google's Material Design 3 guidelines, which is the official design system for modern Android applications. All UI components including buttons, cards, navigation bars, dialogs, and input fields follow Material Design specifications. This ensures the application feels familiar and natural to Android users from the moment they install it.

2.7.2 Simplicity First

CrowdSenseNet is used by ordinary people going about their daily lives. The interface must be simple enough that a non-technical user in Buea, Yaounde, or Bamenda can install the app and start contributing data within 60 seconds. Every design decision prioritises clarity over complexity. Only what is necessary is shown. Everything else is tucked away in settings.

2.7.3 The 3-Tap Rule

No core feature of the application requires more than three taps to reach from the Home screen. This rule prevents the interface from becoming unnecessarily deep or cluttered and keeps the most important actions, which are start session, view map, and check history, always close at hand.

2.7.4 Privacy by Design

Privacy is the top concern of the application's users. The survey found that 56% of respondents listed anonymity as their primary condition for participation. Every design decision reflects this. There is no login screen. There is no user profile. There are no social features. The only identity in the system is an

anonymous UUID token that the user never sees or needs to think about. Privacy is not an afterthought; it is built into the foundation of the app.

2.7.5 Colour-Blind Accessibility

The coverage heatmap uses colour to communicate signal quality. Because colour blindness affects a meaningful portion of the population, the application never relies on colour alone. Every coloured zone on the map also displays a text label, such as Good, Average, Poor, or Insufficient Data, so that users who cannot distinguish between red and green still receive the full information.

2.7.6 Responsive Layout

The interface must render correctly on all Android devices from 5.0 to 6.7 inches without horizontal scrolling or layout breakage. All screens use ConstraintLayout with percentage-based positioning to ensure consistent rendering across the wide variety of Android devices in use across Cameroon.

2.8 App Icon Specification

Specification	Details
Icon Shape	Square with rounded corners following Android Adaptive Icon guidelines
Background	White (#FFFFFF): provides maximum contrast for the colourful icon foreground
Foreground	The map pin with network mesh and signal waves
Primary Colours	Teal (#00BCD4) to Deep Royal Blue (#1565C0) gradient
Home Screen Size	48 x 48 dp (standard Android launcher icon)
Play Store Size	512 x 512 px (required for Google Play Store submission)
Icon Format	Android Adaptive Icon (API 26+) with separate foreground and background layers
File Format	PNG with transparent background for foreground layer

2.9 Conclusion

The app identity for CrowdSenseNet gives the application a clear, professional, and meaningful visual personality. The logo communicates location, connectivity, and intelligence. The color palette is consistent and functional. The typography is readable and accessible. The design principles ensure that the application remains simple, private, and usable for all users across Cameroon. These identity elements form the foundation on which every screen in the application is built.

Jose -Visual Designs

This report documents the visual design work done by Jose for Task 5 of the CEF440 Internet and Mobile Programming project. The visual design covers all five screens of the CrowdSenseNet Android mobile application: the Splash Screen, the Home Screen, the Coverage Map Screen, the History Screen, and the Settings Screen.

All designs were created using Figma, which is a professional browser-based interface design tool used widely in the software industry. Figma allows designers to build high-fidelity mockups that show exactly how an application will look on a real device, including colours, typography, spacing, icons, and component layout.

The purpose of the visual design phase is to define the look and feel of every screen before the code is written. This saves time during development and ensures that all screens follow a consistent visual style. The designs in this report served as the reference that the other group members used when implementing their screens in Kotlin.

The visual design follows the CrowdSenseNet brand identity defined by Amelia, including the color palette, typography system, and Material Design 3 guidelines. Every screen uses the same fonts, spacing rules, and component styles so that the application feels like one unified product.

3. Design Tools and Process

3.1 Tool Used: Figma

Figma was chosen as the design tool for this project for the following reasons:

- It is free to use for students and small teams.
- It runs in the browser, so no installation is required and all team members can view the designs without extra software.
- It supports Android frame sizes natively, making it easy to design screens at the correct dimensions.
- It allows the designer to create reusable components such as buttons, cards, and navigation bars, which keeps the design consistent across all screens.
- It supports prototyping, so screen transitions can be shown to the team before any code is written.

3.2 Design Process

The visual design was completed in three stages. First, a wireframe was sketched for each screen to decide the layout and placement of elements. Second, the wireframes were converted into high-fidelity mockups in Figma using the CrowdSenseNet brand colors and typography. Third, the final mockups were reviewed against the task requirements and shared with the group for use during implementation.

Stage	Description	Output
1. Wireframing	Basic layout sketches showing element placement and screen structure	Low-fidelity wireframes for all 5 screens
2. High-Fidelity Mockup	Full colour designs with real fonts, icons, and component styles applied	Figma mockup files for all 5 screens
3. Review and Handoff	Final check against task requirements and sharing with the group for implementation	Figma share link and exported screen images

3.3 Frame Size

All screens were designed using the standard Android phone frame in Figma. The frame dimensions used were 360 x 800 dp, which represents a common mid-range Android phone screen size. This size is large enough to show all UI elements clearly and small enough to represent the experience on most of the Android devices commonly used in Cameroon.

3.4. Visual Style Guide

Before the individual screen designs are presented, this section documents the visual rules that were applied consistently across all five screens. These rules come from the CrowdSenseNet brand identity defined by Amelia and were applied in Figma to ensure every screen looks like part of the same application.

3.4.1 Color Palette Applied in Figma

Color Name	Hex Code	Where Used in Designs
Deep Royal Blue	#1565C0	Top app bar, bottom navigation bar, primary buttons, active icons
Teal / Cyan	#00BCD4	Accent highlights, signal indicators, card borders, icon accents
White	#FFFFFF	Screen backgrounds, card surfaces, input field backgrounds
Dark Slate	#263238	All body text, labels, and headings across every screen
Light Blue Grey	#ECEFF1	Card backgrounds, list item backgrounds, bottom sheet surfaces
Signal Green	#4CAF50	Good coverage zones on the map, positive status indicators
Amber	#FF9800	Average coverage zones, warning indicators
Danger Red	#F44336	Poor coverage zones, the Delete Data button, error states
Neutral Grey	#9E9E9E	Insufficient data zones, disabled button states, placeholder text

3.4.2 Typography Applied in Figma

Roboto was used as the typeface across all screens in Figma. This matches the Android system font and the Material Design 3 guidelines.

Text Level	Font Style	Size	Where Used
Screen Title	Roboto Bold	24sp	Top app bar title on each screen
Section Header	Roboto Medium	20sp	Card titles and section labels
Signal Values	Roboto Medium	18sp	Live metric numbers on the Home Screen
Body Text	Roboto Regular	16sp	Descriptions, session details, settings labels
Secondary Labels	Roboto Regular	14sp	Sub-labels, list item details, captions
Timestamps and Captions	Roboto Light	12sp	Date and time labels, map zone labels

3.4.3 Component Library

A small component library was built inside Figma before starting the individual screen designs. This ensured that the same button style, card style, and navigation bar were reused on every screen rather than drawn from scratch each time. The main components created were:

- Primary Button: Deep Royal Blue background, white Roboto Medium text, rounded corners with 8dp radius.
- Secondary Button: White background, Deep Royal Blue border and text, same corner radius.
- Destructive Button: Danger Red background, white text. Used only for the Delete Data action.
- Metric Card: White card with Light Blue Grey background, a teal accent border on the left side, a bold metric value at the top, and a small label below.
- Session Card: White card used in the History Screen, containing date, duration, reading count, and upload status fields.
- Bottom Navigation Bar: Deep Royal Blue background with white icons and labels. Active tab shown with a teal underline.
- Top App Bar: Deep Royal Blue background, white screen title in Roboto Bold 24sp, optional back arrow icon on secondary screens.

3.5. Splash Screen Design

3.5.1 Purpose

The Splash Screen is the first thing a user sees when they open CrowdSenseNet. It appears for about two seconds while the application loads in the background. Its purpose is to make a good first impression and confirm to the user that the app is starting correctly. It also gives the application a moment to initialise its database and check the device state before showing the Home Screen.

3.5.2 Layout Description

The Splash Screen has a white background. The CrowdSenseNet logo is placed in the center of the screen. Directly below the logo is the app name, CrowdSenseNet, written in Roboto Bold 32sp in Dark Slate color. Below the app name is the primary tagline written in Roboto Regular 16sp in a lighter grey color. At the very bottom of the screen, a small loading indicator shows that the app is starting.

- Background color: White (#FFFFFF).
- Logo: The official CrowdSenseNet map pin logo centered on screen.
- App name: CrowdSenseNet in Roboto Bold 32sp, Dark Slate (#263238).
- Tagline: Your phone. Your network. Your map. in Roboto Regular 16sp.
- Loading indicator: Small circular progress bar in Deep Royal Blue at the bottom.

Figure 2: Splash Screen High-Fidelity Figma Mockup (splash_screen_design.png)

3.5.3 Design Decisions

The Splash Screen was kept very simple on purpose. A splash screen should not try to do too much. Its job is to show the brand and disappear quickly. Adding too many elements would slow the user down and make the app feel heavy. The white background was chosen to match the app icon background so that the transition from the app icon to the splash screen feels smooth.

3.6. Home Screen Design

3.6.1 Purpose

The Home Screen is the main control screen of CrowdSenseNet. It is where the user starts and stops data collection sessions and monitors the live signal metrics from their device. The design must communicate a lot of information clearly without making the screen feel crowded or difficult to read.

3.6.2 Layout Description

The Home Screen uses a vertical single-column layout divided into four main sections: the top app bar, the GPS status card, the live metrics panel, and the session control area at the bottom.

Section	Position on Screen	Content
Top App Bar	Top of screen	Screen title: Home. Deep Royal Blue background.
GPS Status Card	Below app bar	GPS icon, status text, and coordinate display.
Live Metrics Panel	Center of screen	Four metric cards: RSRP, RSRQ, SINR, RSSI.
Session Status Display	Below metrics	Session duration, reading count, upload status.
Start / Stop Buttons	Bottom of screen	Large Start button (green) and Stop button (red).
Bottom Navigation Bar	Very bottom	Home, Map, History, Settings tabs.

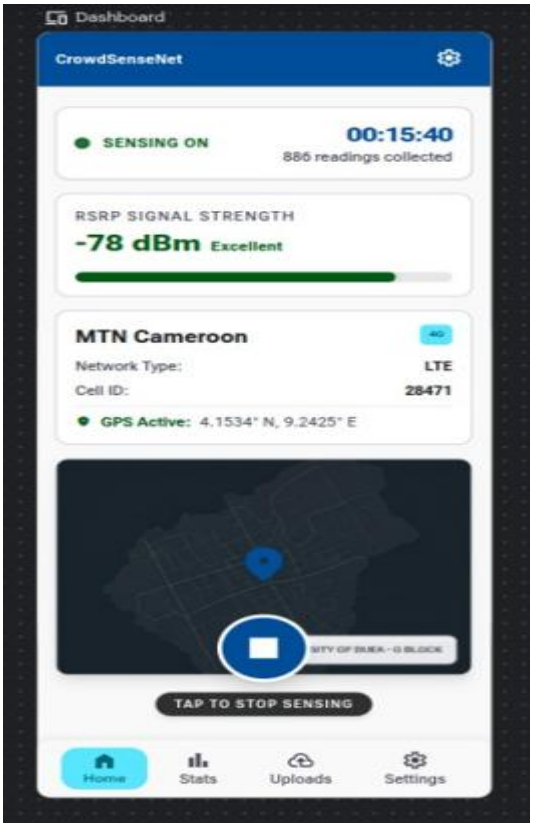

Figure 3: Home Screen High-Fidelity Figma Mockup (home_screen_design.png)

3.6.3 GPS Status Card Design

The GPS status card is placed at the top of the content area, just below the app bar. It uses a white card with a Light Blue Grey background and a teal left border. The card shows a GPS icon on the left, a status label in the center, and the current coordinates below the label in smaller text. Three visual states were designed:

- GPS Active with Fix: teal GPS icon, green status text reading GPS Active, coordinates shown.
- GPS Searching: amber GPS icon, amber status text reading Searching for GPS Fix.
- GPS Disabled: grey GPS icon, red status text reading GPS Disabled. Enable in Settings.

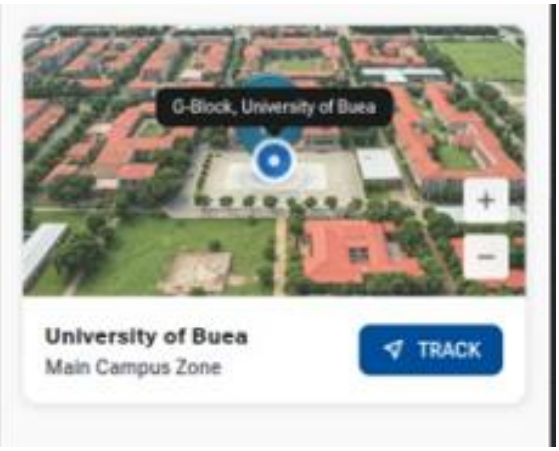

Figure 4: GPS Status Card - Three Design States

3.6.4 Live Metrics Panel Design

The live metrics panel shows four signal values arranged in a 2 by 2 grid of metric cards. Each card has the same structure: a small label at the top showing the metric name, a large bold number in the center showing the current value, and a small unit label below the number. The cards use the Metric Card component from the component library.

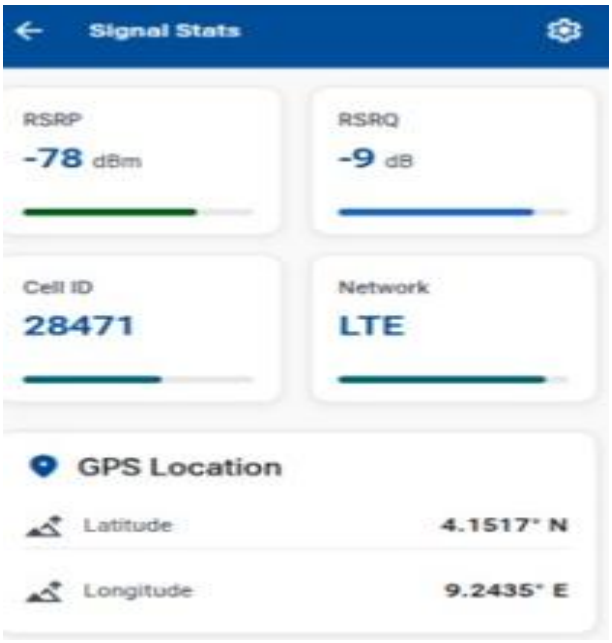

Figure 5: Live Signal Metrics Panel Design

3.6.5 Start and Stop Button Design

The Start button uses the Primary Button component style with a Signal Green background instead of Deep Royal Blue, to make it feel positive and action-inviting. The Stop button uses the Destructive Button

style with a Danger Red background. Both buttons are full-width, taking up the entire content width of the screen, and have large text so they are easy to tap. When one button is active, the other is shown in a disabled state with a lighter, washed-out color.

3.7. Coverage Map Screen Design

3.7.1 Purpose

The Coverage Map Screen shows a heatmap of mobile signal quality in the areas where data has been collected. It is the most visually complex screen in the application and required the most design work. The challenge was to make a technical geographic display simple enough for any user to understand at a glance.

3.7.2 Layout Description

Section	Position on Screen	Content
Top App Bar	Top of screen	Screen title: Coverage Map. Deep Royal Blue background.
Search Bar	Below app bar	White input field with a search icon and placeholder text.
Map Area	Center of screen, large	The heatmap taking up most of the screen height.
Legend Bar	Below map area	Color swatches with labels: Good, Average, Poor, No Data.

Export Button	Above bottom nav bar	Full-width secondary button: Export Coverage Data.
Bottom Navigation Bar	Very bottom	Home, Map, History, Settings tabs. Map tab active.

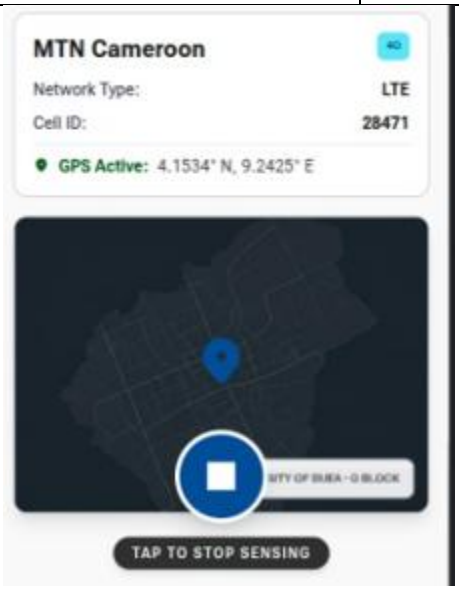

Figure 6: Coverage Map Screen High-Fidelity Figma Mockup (map_screen_design.png)

3.7.3 Heatmap Area Design

The heatmap area occupies the largest portion of the screen. It shows a map of the local area with coloured zones overlaid on top. The four coverage zone colors are taken directly from the brand color palette: Signal Green for good coverage, Amber for average coverage, Danger Red for coverage holes, and Neutral Grey for areas with insufficient data. Each zone is a semi-transparent overlay so that the underlying map roads and landmarks are still visible beneath the color.

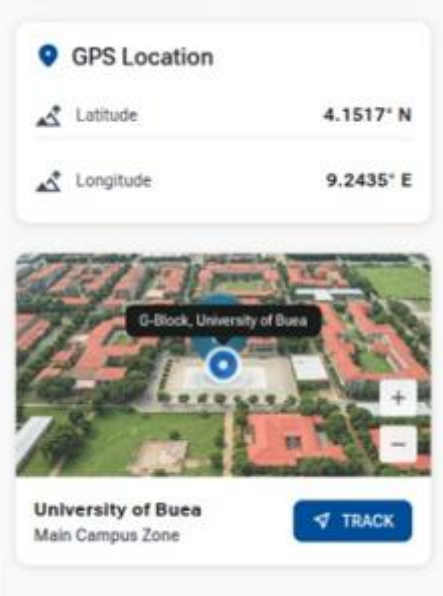

Figure 7: Coverage Heatmap Detail with Zone Colours

3.7.4 Legend Bar Design

The legend bar is a horizontal strip placed directly below the map area. It contains four small coloured squares, each followed by a text label. The labels are: Good Coverage, Average Coverage, Poor Coverage, and No Data. The labels are written in Roboto Regular 12sp in Dark Slate color. The bar has a white background with a thin top border to separate it visually from the map above.

3.7.5 Search Bar and Export Button Design

The search bar sits between the app bar and the map area. It is a full-width white input field with rounded corners and a magnifying glass icon on the left side. The placeholder text reads Search area. The Export Coverage Data button below the map uses the Secondary Button component style with a Deep Royal Blue border and text on a white background.

3.8. History Screen Design

3.8.1 Purpose

The History Screen gives the user a record of all past data collection sessions saved on their device. The design must present a list of sessions in a way that is easy to scan quickly. The user should be able to see the most important information for each session without having to open or expand anything.

3.8.2 Layout Description

Section	Position on Screen	Content
Top App Bar	Top of screen	Screen title: Session History. Deep Royal Blue background.
Session List	Full content area	Scrollable list of session cards in reverse date order.
Empty State	Full content area	Shown when no sessions have been recorded yet.
Bottom Navigation Bar	Very bottom	Home, Map, History, Settings tabs. History tab active.

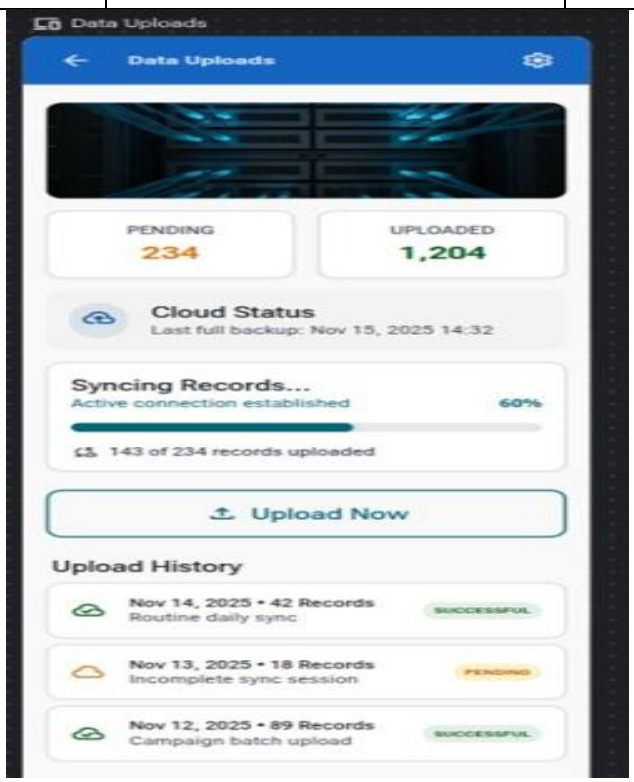

Figure 9: History Screen High-Fidelity Figma Mockup (history_screen_design.png)

3.8.3 Session Card Design

Each session in the list is shown as a Session Card. The card uses the Session Card component from the component library. It has a white background, a Light Blue Grey card surface, and a thin left border in Teal color. The card contains four fields:

- Date and Time: shown at the top of the card in Roboto Medium 14sp. Example: 12/05/2025, 09:34.
- Duration: shown below the date in Roboto Regular 14sp. Example: Duration: 00:23:41.
- Readings: shown on the same line as duration but right-aligned. Example: 142 readings.
- Upload Status: shown at the bottom of the card. Uploaded is shown with a green checkmark icon. Pending Upload is shown with an amber clock icon.

3.8.4 Empty State Design

When the user has not recorded any sessions yet, the session list would be empty. An empty screen with no content looks broken and confuses users. A dedicated empty state design was created for this case. It shows a large grey icon in the center of the screen with a short message below it that reads: No sessions recorded yet. Press Start on the Home Screen to begin collecting data.

3.9. Settings Screen Design

3.9.1 Purpose

The Settings Screen gives the user control over how the application behaves. It contains options for language, data upload rules, WiFi toggle, and data deletion. The design must group related settings together clearly so the user can find what they are looking for without reading every option on the screen.

3.9.2 Layout Description

Section	Position on Screen	Content
Top App Bar	Top of screen	Screen title: Settings. Deep Royal Blue background.
Language Card	First content card	Language dropdown: English (US) and French (FR).
Transmission Rules Card	Second content card	WiFi-only upload toggle and WiFi antenna toggle.
Storage and Admin Card	Third content card	Delete All Data button in Danger Red.
Bottom Navigation Bar	Very bottom	Home, Map, History, Settings tabs. Settings tab active.

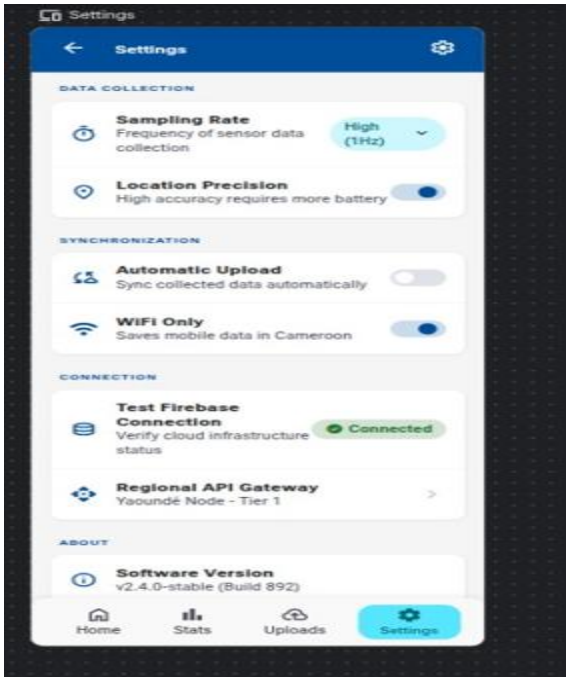

Figure 12: Settings Screen High-Fidelity Figma Mockup (settings_screen_design.png)

3.9.3 Language Card Design

The Language Card is the first settings group. It has a card title of Language at the top in Roboto Medium 16sp. Below the title is a full-width dropdown field showing the currently selected language. The dropdown has a white background with a thin border and a small downward arrow icon on the right side. The two available options are English (US) and French (FR).

3.9.4 Transmission Rules Card Design

The Transmission Rules Card contains two toggle switches. Each toggle is a full-width row with a label on the left and a Material Design toggle switch on the right. The first row is labelled Upload on WiFi Only. The second row is labelled WiFi Enabled. When a toggle is on, the switch shows the Teal accent color. When it is off, the switch shows the Neutral Grey color.

3.9.5 Storage and Administration Card Design

The Storage and Administration Card contains a single button: Delete All Data. This button uses the Destructive Button component with a Danger Red background and white text. The card also contains a short warning label above the button that reads: This will permanently delete all session records from this device. This action cannot be undone. The warning text is written in Roboto Regular 12sp in Danger Red color to reinforce that this is a serious and permanent action.

3.10. Screen Flow and Navigation Design

3.10.1 Overview

One part of the visual design work was creating a screen flow diagram that shows how the five screens connect to each other. This diagram was used by Fernandez as a reference when implementing the NavGraph.kt navigation file. The diagram shows each screen as a box and uses arrows to show which screen leads to which.

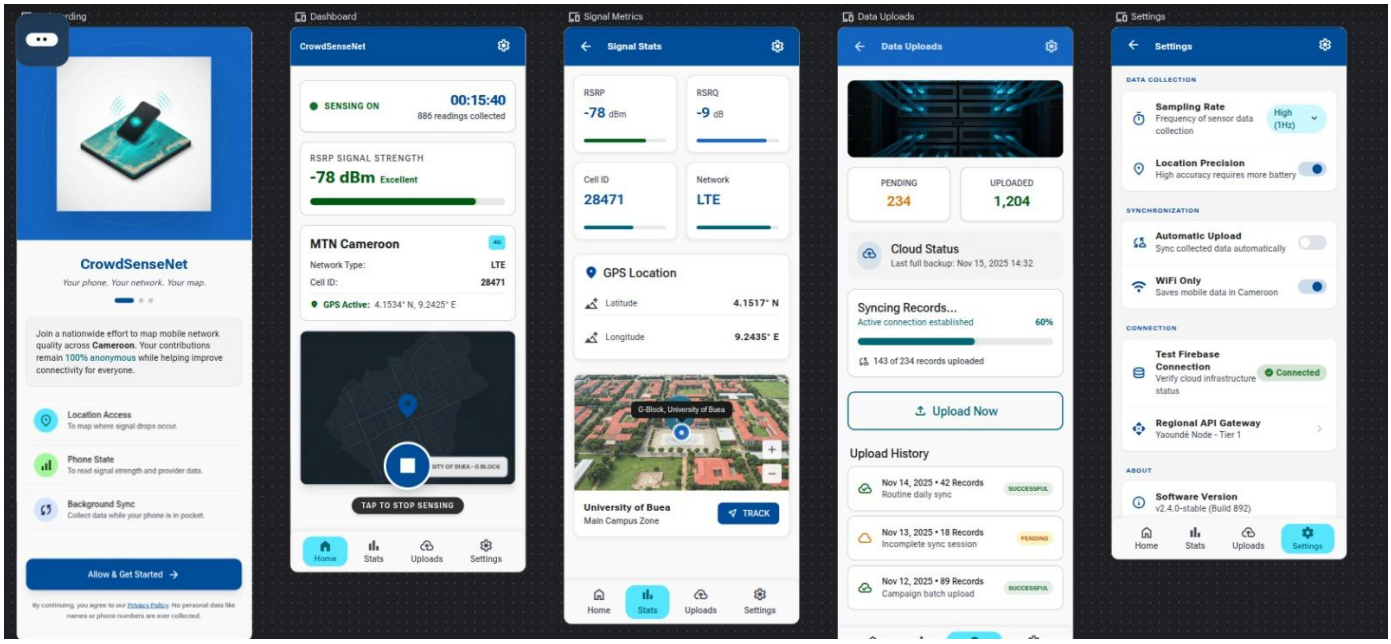

Figure 16: CrowdSenseNet Screen Flow Diagram

3.10.2 Bottom Navigation Bar

The bottom navigation bar appears on all screens except the Splash Screen. It contains four tabs: Home, Map, History, and Settings. The active tab is shown with a white icon and a teal underline. The inactive tabs are shown with slightly dimmer white icons and no underline. The bar has a Deep Royal Blue background consistent with the top app bar.

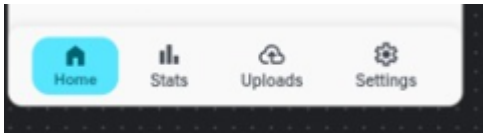

Figure 17: Bottom Navigation Bar - Four Active Tab States

3.11. Expected Output Files

The table below lists all the design files that were produced for the visual design section of Task 5.

File Name	Description
splash_screen_design.png	High-fidelity Figma mockup of the Splash Screen
home_screen_design.png	High-fidelity Figma mockup of the Home Screen
map_screen_design.png	High-fidelity Figma mockup of the Coverage Map Screen
history_screen_design.png	High-fidelity Figma mockup of the History Screen
settings_screen_design.png	High-fidelity Figma mockup of the Settings Screen
figma_component_library.png	Screenshot of the Figma component library
screen_flow_diagram.png	Screen flow diagram showing navigation between all screens
gps_status_card_states.png	Three GPS status card design states
metrics_panel_design.png	Live signal metrics panel design
heatmap_detail.png	Close-up of the coverage heatmap zone colours
legend_bar_design.png	Legend bar design for the Coverage Map Screen
session_card_design.png	Session card design for the History Screen
history_empty_state.png	Empty state design for the History Screen
language_card_design.png	Language settings card design
transmission_card_design.png	Transmission rules card design
storage_card_design.png	Storage and administration card design
bottom_nav_states.png	Bottom navigation bar in four active tab states

3.12. Conclusion

The visual design work done by Jose in Figma provides a complete and consistent visual reference for all five screens of the CrowdSenseNet application. Every screen follows the same brand colors, typography rules, and component styles, which means the application looks and feels like a single unified product rather than five separate screens built independently.

The designs make the implementation work easier and faster because the developers do not need to make any visual decisions during coding. The colors, sizes, spacing, and component styles are all defined in the Figma mockups. The developers simply follow the designs.

The visual design also ensures that CrowdSenseNet meets a professional standard that is appropriate for a final project presentation. The use of Figma, proper component libraries, and Material Design 3 guidelines shows that the design process followed real industry practices.

Frontend Implementation

This report documents the frontend implementation work done by Aubry and Fernandez for Task 5 of the CEF440 Internet and Mobile Programming project. The visual design for all five screens was completed by Jose using Figma, as documented in the Visual Design Report. This report covers the next stage: turning those designs into working Android screens using Kotlin and Jetpack Compose.

Aubry was responsible for implementing the Home Screen and the Coverage Map Screen. Fernandez was responsible for implementing the Settings Screen and the History Screen. The fifth screen, the Splash Screen, was a shared deliverable handled during app setup. Navigation across all screens was also managed by Fernandez as part of the frontend integration work.

All implementation was done on the Android platform using Jetpack Compose for the UI layer, Room for local data storage, and the Android Telephony API for reading live signal metrics. The designs produced by Jose in Figma served as the direct reference for all layout, colour, and typography decisions in the code.

4. Aubry - Home Screen Implementation

4.1 Introduction

The Home Screen is the main control screen of CrowdSenseNet. It is the first screen the user sees after the splash screen and the screen they return to most often. The implementation needed to display live signal metrics, a GPS status indicator, a session status panel, and Start and Stop buttons. All of these elements had to update in real time while a session is active.

4.2 Architecture and ViewModel

The Home Screen follows the MVVM architecture pattern. The UI layer is a Jetpack Compose Composable that observes a HomeViewModel. The ViewModel holds all the screen state as StateFlow objects, which the Composable collects using collectAsState(). This means the UI rebuilds only the parts that change when new data arrives, keeping the screen fast and responsive.

The ViewModel communicates with a background service that reads signal metrics from the Android Telephony API at regular intervals. When a session is active, the service sends new readings to the ViewModel, which updates the StateFlow values and triggers a UI recomposition.

```
class HomeViewModel : ViewModel() {
    private val _metrics = MutableStateFlow(NetworkMetrics(-88, -11, 14))
    val metrics = _metrics.asStateFlow()

    private val _isCollecting = MutableStateFlow(false)
    val isCollecting = _isCollecting.asStateFlow()

    private val _readingsCount = MutableStateFlow(0)
    val readingsCount = _readingsCount.asStateFlow()

    private val _gpsAccuracy = MutableStateFlow("High Accuracy (±3.6m)")
    val gpsAccuracy = _gpsAccuracy.asStateFlow()
```

Figure 1: HomeViewModel.kt - StateFlow declarations

4.3 GPS Status Indicator Implementation

The GPS status indicator is implemented as a Composable that observes the GPS state from the ViewModel. The Composable uses a when expression to switch between three display states based on the

current GPS value: active with fix, searching, and disabled. Each state shows a different icon colour and status text, matching the designs produced by Jose in Figma.

The GPS state is updated by a LocationListener registered in the background service. When a valid GPS fix is obtained, the listener sends the coordinates to the ViewModel. If the device's location service is disabled, the Composable shows a prompt directing the user to the device settings.

Figure 3: Home Screen running on device (home_screen_screenshot.png)

4.4 Live Signal Metrics Display Implementation

The live signal metrics panel is implemented as a LazyVerticalGrid with two columns. Each cell in the grid is a MetricCard Composable. The MetricCard takes a label string and a value string as parameters and displays them in the card layout defined in the Figma design. When no reading is available, the value parameter is passed as N/A so the card never shows a blank field.

The four metric values, which are RSRP, RSRQ, SINR, and RSSI, are read from the Android TelephonyManager using the getAllCellInfo() method. The readings are polled every three seconds while a session is active. On devices running Android 12 and above, the READ_PHONE_STATE permission is required and is requested at runtime before the first reading attempt.

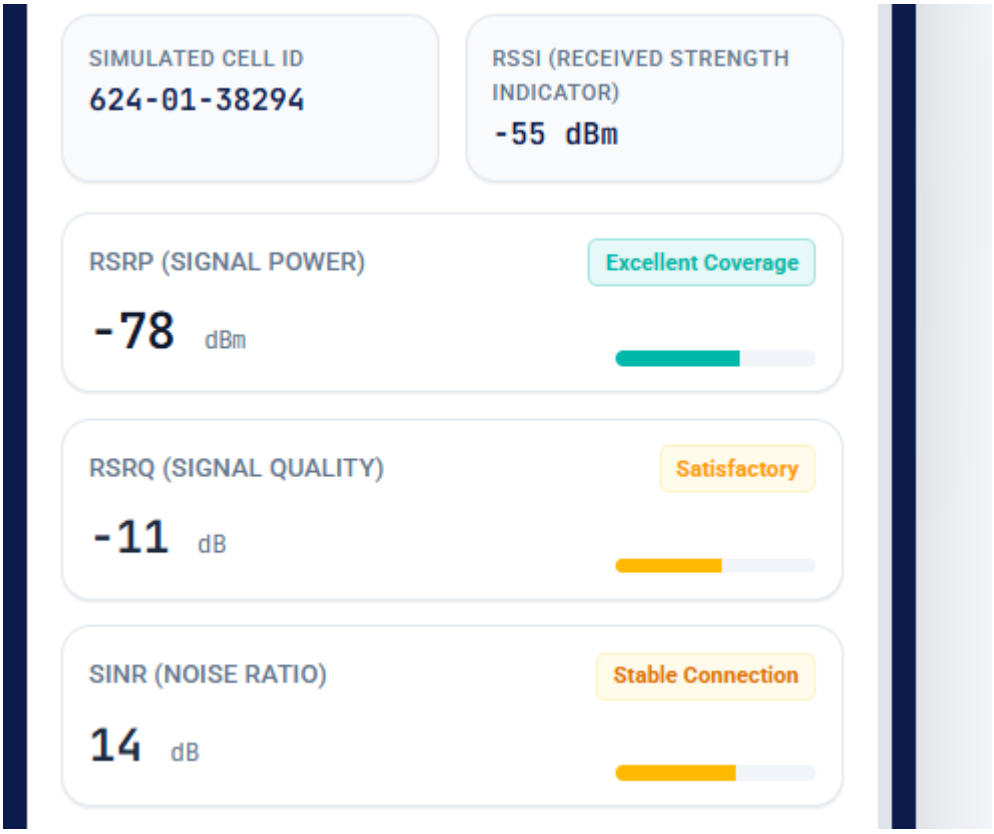

Figure 4: MetricCard Composable and Metrics Grid - HomeScreen.kt

4.5 Start and Stop Button Implementation

The Start and Stop buttons are implemented as full-width Button Composables. The active button uses the primary button style from the Material Design 3 theme. The inactive button is shown in a disabled state using the enabled parameter of the Button Composable, which automatically applies a washed-out appearance following the Material Design disabled state specification.

Pressing the Start button calls the startSession() function on the ViewModel, which starts the background data collection service and sets the session state to active. Pressing the Stop button calls stopSession(), which stops the service, writes the completed session to the Room database, and resets all metric values to their idle state.

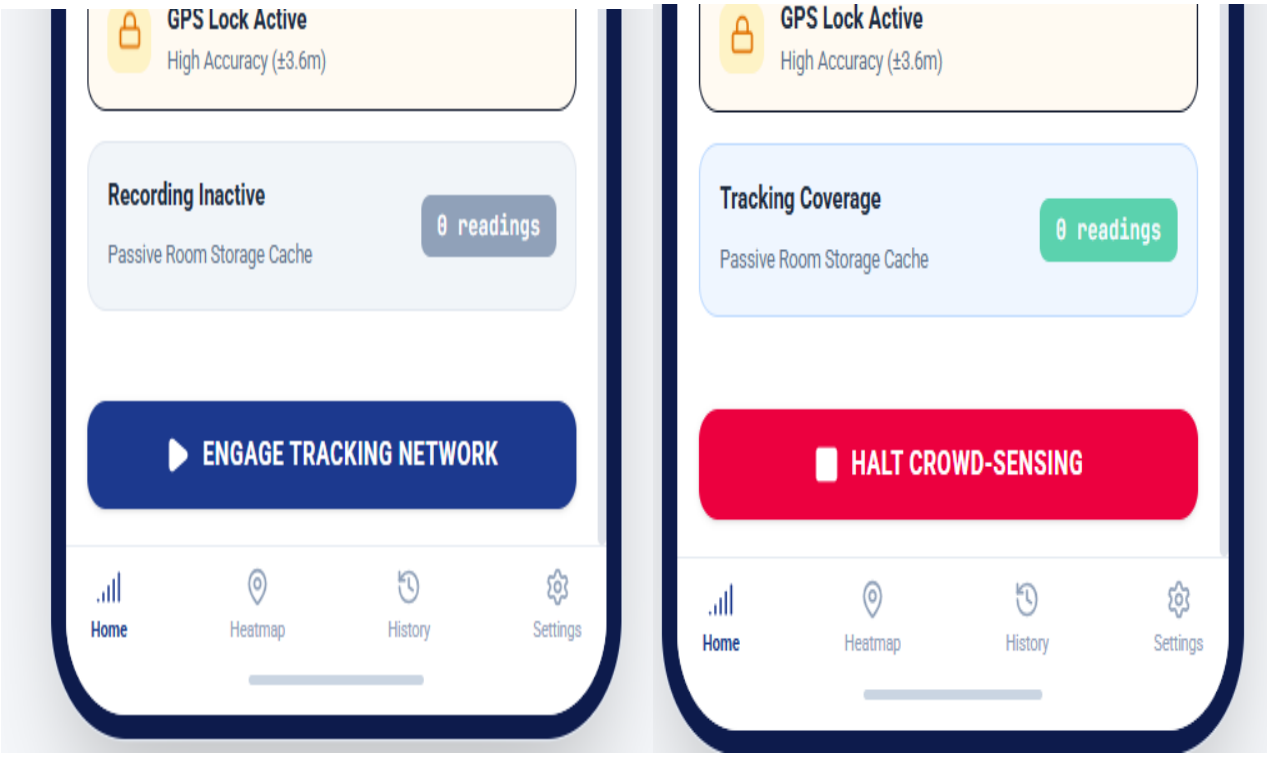

Figure 5: Start and Stop Button Implementation - HomeScreen.kt

4.6 Session Status Panel Implementation

The session status panel is a Card Composable that shows the current state of an active session. It observes four values from the ViewModel: session duration formatted as HH:MM:SS, reading count as an integer, current GPS coordinates as a formatted string, and upload status as an enum with two values: PENDING and UPLOADED.

When no session is active, the panel shows a single line of text telling the user to press Start to begin. This idle state prevents the panel from looking empty or broken when the app is first opened.

4.7 Expected Output Files

File	Description
HomeScreen.kt	Kotlin source file for the Home Screen composable and all sub-composables
HomeViewModel.kt	ViewModel managing state for the Home Screen

home_screen_screenshot.png	Screenshot of the running Home Screen on device

4.8 Conclusion

The Home Screen implementation gives the user full control over data collection sessions with live feedback on signal quality and GPS state. The MVVM architecture keeps the UI clean and the logic separated into the ViewModel. The implementation follows the Figma design produced by Jose closely, with the same card layout, colors, and button styles applied in Compose.

5. Aubry - Coverage Map Screen Implementation

5.1 Introduction

The Coverage Map Screen is the most visually complex screen in the application. It displays a heatmap of mobile signal coverage overlaid on a real map, with a legend, a search bar, and an export button. The implementation required integrating a mapping library with the Jetpack Compose UI layer and writing logic to render coloured zones on top of the map based on signal data stored in the local database.

5.2 Map Integration

The map is rendered using the Google Maps Compose library, which provides a GoogleMap Composable that integrates directly with the Jetpack Compose layout system. The map is initialised with a default camera position centered on Cameroon. When signal data is available in the local database, the map overlays coloured polygons on top of the base map tiles to show the coverage zones.

```
class MapViewModel : ViewModel() {
    private val _heatmapPoints = MutableStateFlow<List<GridCoordinate>>(emptyList())
    val heatmapPoints = _heatmapPoints.asStateFlow()

    private val _searchQuery = MutableStateFlow("")
    val searchQuery = _searchQuery.asStateFlow()

    private val _exportResult = MutableStateFlow<String?>(null)
    val exportResult = _exportResult.asStateFlow()

    init {
        // Build diagnostic signal matrix for preloaded points
        loadCoverageData("default_center")
    }

    fun loadCoverageData(area: String) {
        viewModelScope.launch {
            // Simulated retrieval of network geo-reads matching query
            _heatmapPoints.value = listOf(
                GridCoordinate(48.8566, 2.3522, -78),  // Green
                GridCoordinate(48.8584, 2.2945, -92),  // Yellow
                GridCoordinate(48.8606, 2.3376, -105), // Red
                GridCoordinate(48.8534, 2.3488, -118), // Black (Dead zone)
                GridCoordinate(48.8492, 2.3265, -82)   // Green
            )
        }
    }
```

Figure 6: GoogleMap Composable Setup - MapScreen.kt

5.3 Heatmap Rendering

The heatmap is rendered by drawing a set of filled polygons on the map. Each polygon represents a geographic zone that has been assigned a coverage class based on the average RSRP value of all signal readings collected within that area. The four coverage classes and their corresponding colours are taken directly from the brand identity defined by Amelia:

Coverage Class	RSRP Threshold	Polygon Colour	Hex Code
Good Coverage	RSRP > -80 dBm	Signal Green	#4CAF50
Average Coverage	RSRP -80 to -100 dBm	Amber	#FF9800
Coverage Hole	RSRP < -110 dBm	Danger Red	#F44336
Insufficient Data	< 50 readings per 1 km2	Neutral Grey	#9E9E9E

Each polygon is drawn with a semi-transparent fill at 60% opacity so that the underlying map roads and landmarks remain visible beneath the colour overlay. The polygon boundaries are computed by grouping signal readings into 1 km by 1 km grid cells and using the bounding box of each cell as the polygon shape.

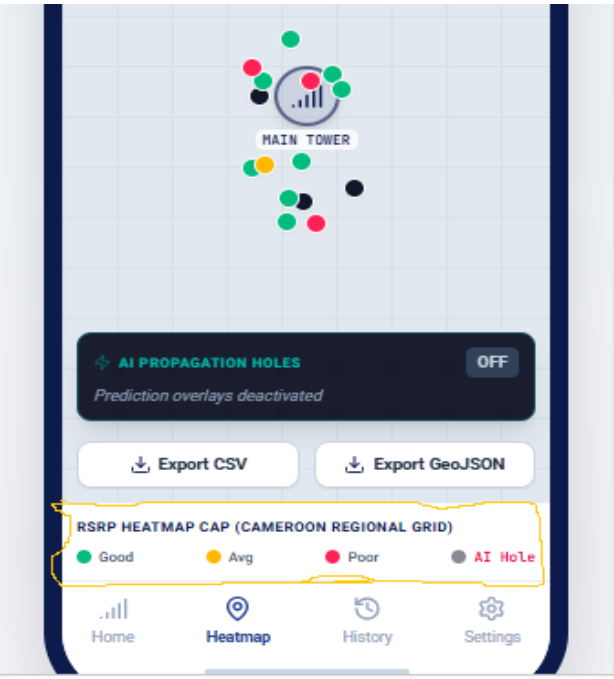

Figure 7: Heatmap Polygon Rendering Logic - MapScreen.kt

Figure 8: Coverage Map Screen running on device (map_screen_screenshot.png)

5.4 Legend Display Implementation

The legend is implemented as a Row Composable placed in a Card below the map area. Each entry in the legend is a small Box Composable filled with the zone colour followed by a Text Composable with the zone label. The four entries are arranged horizontally with equal spacing. The legend is always visible on screen and does not scroll with the map.

5.5 Search Area Functionality Implementation

The search bar is implemented as a TextField Composable at the top of the screen below the app bar. When the user types a location name and presses the search button on the keyboard, the app uses the Android Geocoder API to convert the location name into GPS coordinates. The map camera then animates to the new coordinates using the animateCamera() function on the CameraPositionState, with a zoom level set to show a 10 km radius around the searched location.

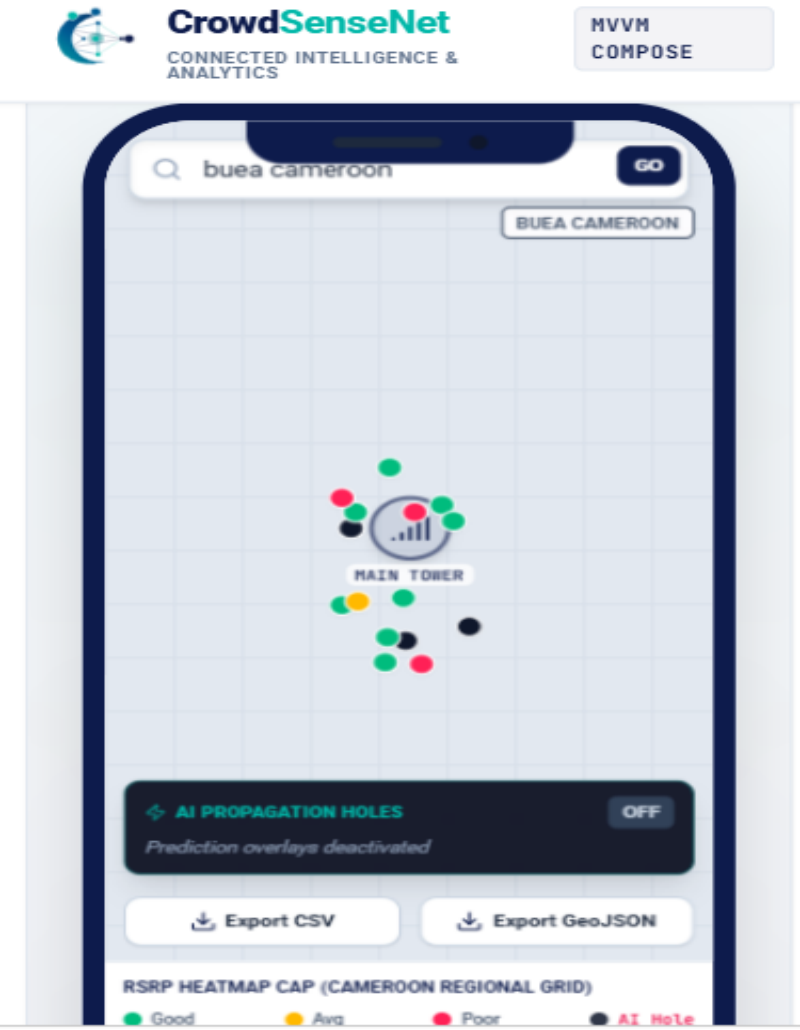

Figure 9: Search Bar and Geocoder Implementation - MapScreen.kt

5.6 Export Button Implementation

The export button triggers a function that reads all signal readings from the local Room database and writes them to a CSV file in the device's external storage. The CSV file contains one row per reading with the following columns: timestamp, latitude, longitude, RSRP, RSRQ, SINR, RSSI, and cell ID. Once the file is written, the app launches an Android share intent so the user can choose to send the file by email, upload it to Google Drive, or open it with another app.

Figure 10: Export Button Implementation - MapScreen.kt

5.7 Expected Output Files

File	Description
MapScreen.kt	Kotlin source file for the Coverage Map Screen composable
MapViewModel.kt	ViewModel managing coverage data and map state
map_screen_screenshot.png	Screenshot of the running Coverage Map Screen on device

5.8 Conclusion

The Coverage Map Screen implementation transforms raw signal readings stored in the local database into a clear visual heatmap that any user can understand. The integration of Google Maps Compose, the polygon rendering logic, the search bar, and the export function together produce a screen that is both useful for ordinary users and professional enough for engineering analysis. The implementation follows the Figma design produced by Jose, with the same colour coding, legend layout, and button styles.

6. Fernandez - Settings Screen Implementation

6.1 Introduction

The Settings Screen gives the user control over how CrowdSenseNet behaves. Fernandez implemented the Settings Screen, the History Screen, and the central navigation system that connects all five screens. This section covers the Settings Screen implementation. The History Screen and navigation are covered in Sections 5 and 6.

6.2 Architecture and ViewModel

The Settings Screen follows the same MVVM pattern used by the other screens. A SettingsViewModel manages the current state of all settings values as StateFlow objects. The settings are stored persistently using Android DataStore, which is the modern replacement for SharedPreferences recommended by Google. DataStore stores key-value pairs on the device and survives app restarts, so the user's settings are remembered between sessions.

```
29  class SettingsViewModel : ViewModel() {
30      private val _language = MutableStateFlow("English")
31      val language = _language.asStateFlow()
32
33      private val _wifiOnly = MutableStateFlow(true)
34      val wifiOnly = _wifiOnly.asStateFlow()
35
36      private val _isWifiEnabled = MutableStateFlow(true)
37      val isWifiEnabled = _isWifiEnabled.asStateFlow()
38
39      private val _dbStatusMessage = MutableStateFlow<String?>(null)
40      val dbStatusMessage = _dbStatusMessage.asStateFlow()
41
42      fun setLanguage(lang: String) {
43          _language.value = lang
44      }
45
46      fun setUploadPreference(pref: Boolean) {
47          _wifiOnly.value = pref
48      }
```

Figure 11: SettingsViewModel.kt - DataStore and StateFlow setup

6.3 Language Selection Implementation

The language selection is implemented as an ExposedDropdownMenuBox Composable, which is the Material Design 3 dropdown component for Jetpack Compose. The dropdown shows two options: English (US) and French (FR). When the user selects an option, the new value is written to DataStore via the ViewModel and the app's string resources are switched to the matching language using the Android locale configuration system.

A bilingual string map is maintained in the ViewModel that holds translations for every label, button text, and message in the app. When the language changes, the map is updated and all Composables that observe these strings recompose with the new language. This approach avoids a full app restart when switching language.

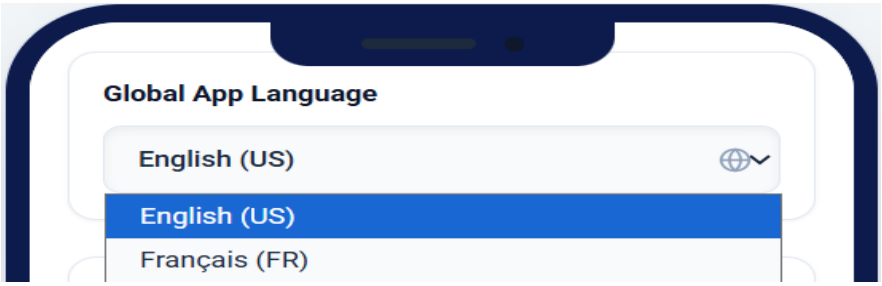

Figure 12: Language Dropdown Implementation - SettingsScreen.kt

```
DropdownMenu(
    expanded = showLangSelector,
    onDismissRequest = { showLangSelector = false }
) {
    DropdownMenuItem(
        text = { Text("English (US)") },
        onClick = {
            viewModel.setLanguage("English")
            showLangSelector = false
        }
    )
    DropdownMenuItem(
        text = { Text("Français (FR)") },
        onClick = {
            viewModel.setLanguage("Français")
            showLangSelector = false
        }
    )
}
```

Figure 13: (settings_screen_screenshot.png)

6.4 Upload Settings and WiFi Toggle Implementation

The two upload toggles are implemented as Switch Composables inside a Card. Each Switch observes its corresponding Boolean StateFlow from the ViewModel. When the user flips a toggle, the new value is

saved to DataStore via the ViewModel. The upload service checks both values before attempting a sync: if WiFi-only upload is enabled, the service checks the network connectivity state before uploading. If the WiFi antenna toggle is off, the upload is blocked regardless of the connection type.

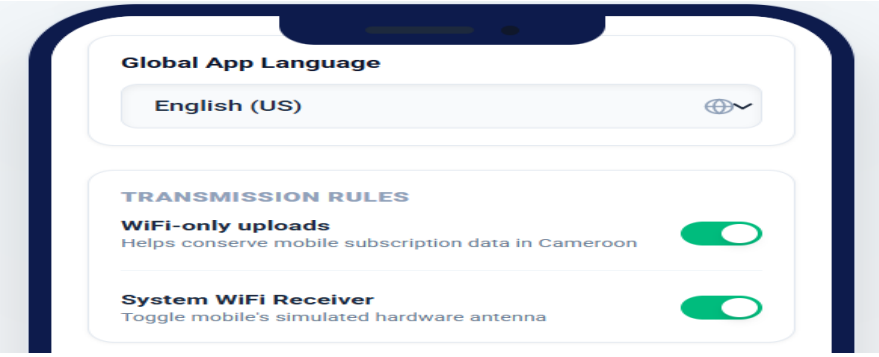

Figure 14: WiFi Toggle Switch Implementation - SettingsScreen.kt

6.5 Delete Data Option Implementation

The Delete All Data button is implemented as a Button Composable with the containerColor set to Danger Red (#F44336) from the brand palette. Pressing the button does not delete data immediately. Instead, it shows a confirmation dialog built with AlertDialog Composable. The dialog asks the user to confirm the action before proceeding. Only after the user confirms does the ViewModel call the Room database deleteAllSessions() function, which wipes all stored session records. After deletion, the History Screen list is automatically updated because it observes the same Room database as a Flow.

Figure 15: Delete Data Button and Confirmation Dialog - SettingsScreen.kt

6.6 Expected Output Files

File	Description
SettingsScreen.kt	Kotlin source file for the Settings Screen composable

SettingsViewModel.kt	ViewModel managing settings state and DataStore persistence
settings_screen_screenshot.png	Screenshot of the running Settings Screen on device

4.7 Conclusion

The Settings Screen implementation gives the user persistent control over the application's behaviour. Language, upload rules, and data management are all handled cleanly through the ViewModel and DataStore. The confirmation dialog for data deletion prevents accidental data loss. The implementation follows the Figma design from Jose, with the three grouped settings cards, the correct toggle styles, and the Danger Red button colour all applied correctly in Compose.

7. Fernandez - History Screen Implementation

7.1 Introduction

The History Screen displays a list of all past data collection sessions stored on the device. Each session card shows the date, duration, reading count, and upload status for that session. The implementation required reading session records from the local Room database and presenting them in a scrollable list that updates automatically when new sessions are added or when upload status changes.

7.2 Architecture and ViewModel

The History Screen uses a HistoryViewModel that exposes a Flow of session records from the Room database. The Composable collects this Flow using collectAsLazyPagingItems() from the Paging 3 library. This means the list only loads a page of sessions at a time rather than loading every session record into memory at once, which keeps the screen fast even when hundreds of sessions have been recorded.

```
35  class HistoryViewModel : ViewModel() {
36      private val _sessions = MutableStateFlow<List<SessionLog>>(emptyList())
37      val sessions = _sessions.asStateFlow()
38
39      init {
40          getSessionHistory()
41      }
42
43      fun getSessionHistory() {
44          viewModelScope.launch {
45              // Simulated Room Entity fetch
46              _sessions.value = listOf(
47                  SessionLog("1", "June 7, 2026 11:15 AM", "12 mins", 124, true),
48                  SessionLog("2", "June 6, 2026 03:40 PM", "8 mins", 82, true),
49                  SessionLog("3", "June 5, 2026 09:20 AM", "25 mins", 310, false),
50                  SessionLog("4", "June 3, 2026 01:10 PM", "4 mins", 41, true),
51                  SessionLog("5", "May 30, 2026 06:05 PM", "15 mins", 158, false)
52              )
```

Figure 16: HistoryViewModel.kt - Room database Flow setup

7.3 Session List Implementation

The session list is implemented as a LazyColumn Composable. Each item in the list is a SessionCard Composable that receives a SessionEntity object and displays its fields. The list is sorted in reverse chronological order by the session start timestamp so the most recent session always appears first. This is handled in the Room database query using an ORDER BY clause rather than sorting in the UI layer.

```
44          viewModelScope.launch {
45              // Simulated Room Entity fetch
46              _sessions.value = listOf(
47                  SessionLog("1", "June 7, 2026 11:15 AM", "12 mins", 124, true),
48                  SessionLog("2", "June 6, 2026 03:40 PM", "8 mins", 82, true),
49                  SessionLog("3", "June 5, 2026 09:20 AM", "25 mins", 310, false),
50                  SessionLog("4", "June 3, 2026 01:10 PM", "4 mins", 41, true),
51                  SessionLog("5", "May 30, 2026 06:05 PM", "15 mins", 158, false)
52              )
```

Figure 17: Session List LazyColumn - HistoryScreen.kt

Figure 18: History Screen running on device (history_screen_screenshot.png)

7.4 Session Card Design Implementation

The SessionCard Composable displays four pieces of information for each session:

- Date and time: formatted as DD/MM/YYYY HH:MM using the SimpleDateFormat class applied to the session start timestamp.
- Duration: calculated by subtracting the start timestamp from the end timestamp and formatting the result as HH:MM:SS.
- Reading count: displayed as a plain integer pulled directly from the SessionEntity.
- Upload status: displayed using an Icon Composable and a Text Composable. A green checkmark icon is shown for uploaded sessions. An amber clock icon is shown for sessions that are still pending upload.

```
@Composable
fun HistoryScreen(viewModel: HistoryViewModel = viewModel()) {
    val sessions by viewModel.sessions.collectAsState()

    Scaffold(
        topBar = {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .background(DeepBlue)
                    .padding(16.dp)
            ) {
                Text(
                    text = "Telemetry History",
                    color = Color.White,
                    fontSize = 20.sp,
                    fontWeight = FontWeight.Bold
                )
                Text(
                    text = "Room DB Repository",
                    color = LightBackground.copy(alpha = 0.7f),
                    fontSize = 12.sp
                )
            }
```

Figure 19: SessionCard Composable - HistoryScreen.kt

7.5 Empty State Implementation

When the Room database contains no session records, the LazyColumn would display an empty list, which looks broken. A dedicated empty state is shown in this case. The Composable checks whether the paged list is empty after loading and switches to an empty state layout if true. The empty state shows a centered icon and a short message telling the user to go to the Home Screen and press Start to begin collecting data.

7.6 Expected Output Files

File	Description
HistoryScreen.kt	Kotlin source file for the History Screen composable
HistoryViewModel.kt	ViewModel managing the Room database session list Flow
history_screen_screenshot.png	Screenshot of the running History Screen on device
history_empty_state_screenshot.png	Screenshot of the History Screen empty state on device

7.7 Conclusion

The History Screen implementation gives the user a clear and automatically updated record of all past sessions. The use of Room with a Flow and Paging 3 ensures the list is efficient regardless of how many sessions are stored. The session card layout matches the Figma design from Jose, and the empty state prevents a confusing blank screen for first-time users.

8. Fernandez - Navigation Integration

8.1 Overview

All screen routing in CrowdSenseNet is managed centrally through a single file called NavGraph.kt. This file defines all the navigation destinations in the app and connects them together using the Jetpack Compose Navigation library. Fernandez was responsible for building and maintaining this file as part of the frontend integration work.

8.2 Screen Destinations

The Screen class defines five navigation destinations as a sealed class. Each destination has a route string, a display title, and an optional bottom navigation bar icon. The five destinations are:

Destination	Route String	Bottom Nav Icon	Bottom Nav Visible
Splash	splash	None	No
Home	home	Home icon	Yes
Map	map	Map icon	Yes
History	history	History icon	Yes
Settings	settings	Settings icon	Yes

```
sealed class Screen(val route: String, val title: String, val icon: ImageVector?) {
    object Splash : Screen("splash", "Welcome", null)
    object Home : Screen("home", "Home", Icons.Default.Home)
    object Map : Screen("map", "Heatmap", Icons.Default.Map)
    object History : Screen("history", "History", Icons.Default.History)
    object Settings : Screen("settings", "Settings", Icons.Default.Settings)
}
```

Figure 21: NavGraph.kt - Screen destinations and NavHost

8.3 Bottom Navigation Bar Implementation

The bottom navigation bar is implemented as a NavigationBar Composable in the main activity. It renders on all screens except the Splash Screen. The active route is determined by observing the current back stack entry from the NavController. The active tab is highlighted with the teal accent color. Tapping a tab calls navController.navigate() with the corresponding route, using launchSingleTop set to true to prevent duplicate destinations on the back stack.

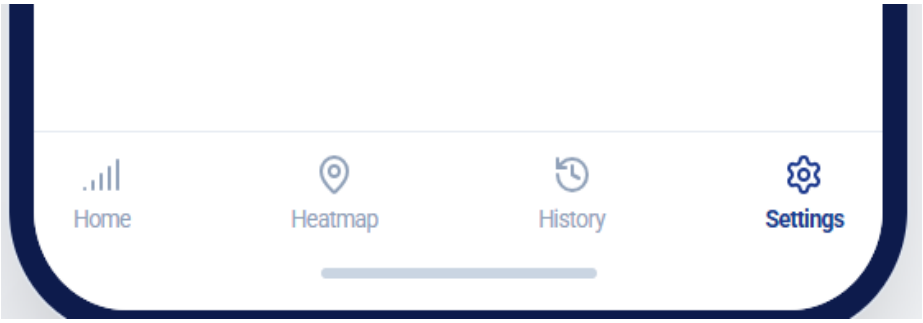

Figure 22: Bottom Navigation Bar Implementation - NavGraph.kt

8.4 Expected Output Files

File	Description
NavGraph.kt	Kotlin source file defining all screen destinations and the NavHost
MainActivity.kt	Main activity hosting the NavHost and the bottom navigation bar

8.5 Conclusion

The centralised navigation system ties all five screens together into a single coherent application. By managing all routing in one file, the codebase is easy to understand and maintain. The bottom navigation bar gives the user instant access to all main screens from anywhere in the app. The implementation follows the screen flow diagram produced by Jose in the visual design phase.

9. Honesty Report

9.1 Purpose

This honesty report documents the actual contributions made by each member of the group for Task 5. It is intended to give an accurate account of who did what, so that the assessment reflects individual effort fairly.

9.2 Individual Contribution Summary

Member Name	Role / Section	Member Name	Role / Section
Amelia	**App Identity and Branding**	**Amelia**	**App Identity and Branding**
Jose	**Visual Design (Figma)**	**Jose**	**Visual Design (Figma)**
Aubry	**Home, Coverage Map Screen UI and Frontend Integration**	**Aubry**	**Home, Coverage Map Screen UI and Frontend Integration**
Fernandez	**Settings, History Screen and Frontend Integration**	**Fernandez**	**Settings, History Screen and Frontend Integration**
Bessong	**……..**	**Bessong**	**……..**

10. Conclusion

Task 5 required the group to design and implement the complete user interface layer of the CrowdSenseNet Android mobile application. The work was divided into three distinct parts: visual design, frontend implementation, and app identity and branding.

Amelia established the visual foundation of the application by defining the app name, logo, color palette, typography system, and design principles. Her work gave the entire project a consistent and professional brand identity that every other member referenced throughout their own work.

Jose translated that brand identity into concrete screen designs using Figma. He produced high-fidelity mockups for all five screens, a reusable component library, and a screen flow diagram. These designs served as the direct reference for the developers and ensured that all screens would look like parts of a single unified application before a single line of code was written.

Aubry implemented the Home Screen and the Coverage Map Screen in Kotlin using Jetpack Compose. The Home Screen gives the user live signal feedback and full control over data collection sessions. The Coverage Map Screen transforms raw signal readings from the local database into a colour-coded heatmap that any user can understand at a glance, with search and export functionality added for professional use.

Fernandez implemented the Settings Screen and the History Screen, and built the central navigation system that connects all five screens together. The Settings Screen gives the user persistent control over language and upload behaviour. The History Screen provides a clear, automatically updated record of all past sessions. The NavGraph.kt file ties the entire application together into one coherent product.

Together, the three parts produced a complete, functional, and visually consistent UI layer for CrowdSenseNet. The application is simple enough for an ordinary user in Cameroon to use without any training, and professional enough to serve as a working prototype for real network coverage analysis.